

FDE SUBMISSION REPORT

EMI Flow

EMI Payment Collection Agent

Architecture, implementation, mandatory tests, demonstration evidence, limitations, and production-readiness plan

MY DEMONSTRATION

I built and deployed the application lifecycle end to end using an explicitly labeled mock Gnani trigger plus authenticated simulated post-call webhooks. I also completed and recorded a real multi-turn Gnani Agent Console conversation before my Agent Console call credits were exhausted. Provider-triggered outbound calling remains dependent on Gnani-issued trigger credentials and credits.

Prepared by Vaishnavi Venkateswaran

28 July 2026

1. Executive summary

I built and deployed EMI Flow as a FastAPI, React, and MongoDB Atlas solution for capturing EMI collection call requests, storing pending call state, processing authenticated and idempotent post-call outcomes, and displaying stage codes, disposition reasons, transcripts, and operational totals.

Item	Value
Live dashboard	https://emi-flow-dashboard.onrender.com
Live API	https://emi-flow-api.onrender.com
API documentation	https://emi-flow-api.onrender.com/docs
Source repository	https://github.com/vaishnaviikv/EMI-payment-collection-agent
Gnani agent ID	bce1ae6dde324dca8aba045f2c86f1fc
Live conversation ID	9b8a1885-efd3-4d9a-9fdc-367de019e116
Database	MongoDB Atlas - emi_flow.calls
Deployment	Render Web Service + Render Static Site
Free-tier demo note	My Render services can spin down after inactivity. Before a live demonstration, I deploy the latest commit to restore a fresh live state, then refresh the dashboard; the first backend request may still take about 50 seconds.

Evidence legend

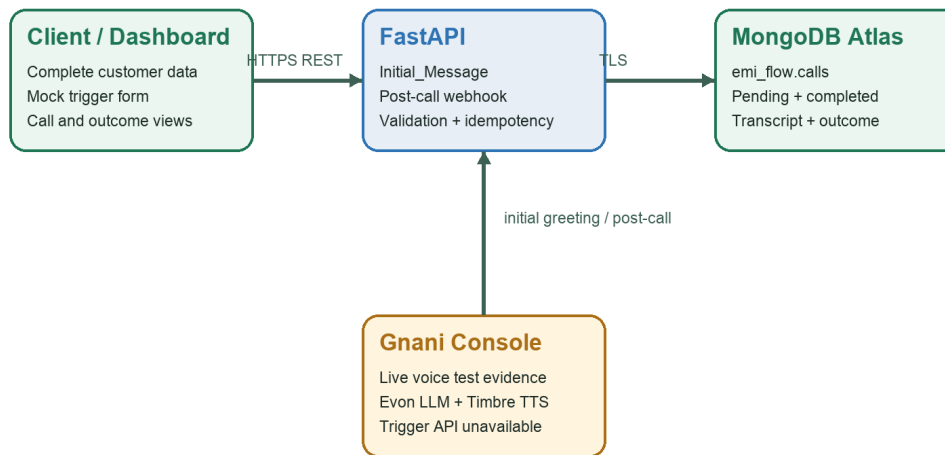
PASS

Implemented and supported by captured evidence, automated tests, or visible deployed behavior.

PARTIAL

A meaningful portion is demonstrated, but exact provider behavior or the required named model/version is not fully evidenced.

2. Architecture and lifecycle



Demo mode replaces the unavailable provider trigger with a mock provider ID; later API, persistence, and dashboard behavior remains real.

Figure 1. Deployed architecture and documented mock-provider boundary.

- Initial request: the dashboard or client sends complete customer and EMI data to POST `/api/Initial_Message`.
- Persistence: FastAPI validates and inserts a pending call record in MongoDB Atlas.
- Provider boundary: mock mode returns a deterministic mock provider ID because the current account exposes no outbound-trigger URL or credentials.
- Post-call: an authenticated webhook applies transcript, outcome analytics, stage code, and disposition reason atomically.
- Presentation: the dashboard reads persisted records through FastAPI and supports search and outcome filters.

3. Demonstration expectations

#	Expectation	Status	Evidence / qualification
1	Explain overall architecture	PASS	Architecture diagram, deployed URLs, API/Mongo/dashboard lifecycle.
2	Show Gnani Voice Agent configuration	PASS	Agent details, languages, Evon model, Timbre voice, analytics fields, conversation logs.
3	Confirm Prisma ASR, Timbre 2.5 TTS, Evon LLM	PARTIAL	Prisma v2.5 is visible in the Gnani API playground and Evon v2.0 Fast is visible in agent settings. The agent uses Timbre G v1.0 because it was the only TTS model available in this Agent Console account; Timbre 2.5 was not offered. The agent Transcriber selection was not captured.
4	Initiate a call using FastAPI	PASS	Create test call posts complete FDE data; FastAPI validates, stores, and returns triggered status plus mock provider ID.
5	Demonstrate multi-turn conversation	PASS	Real Gnani console recording and transcript show a 1m08s multi-turn voice test; simulated records add scenario-specific conversations.
6	Show post-call webhook received	PARTIAL	Authenticated simulated webhook is proven through completed records and <code>webhook_ids</code> . Exact automatic Gnani post-call JSON was not captured.

#	Expectation	Status	Evidence / qualification
7	Show database record updated	PASS	MongoDB screenshot shows completed state, webhook ID, transcript, outcome, disposition, and provider ID.
8	Show stage code and disposition reason	PASS	Dashboard call detail and mandatory scenario overview show normalized outcomes and reason.
9	Demonstrate failure handling	PASS	Duplicate delivery, invalid request, missing-call recovery, mocked trigger failure, and timeout paths are implemented and automated.
10	Explain production readiness	PASS	Section 11 covers credentials, provider contract, security, observability, retries, privacy, and deployment hardening.

4. Mandatory test scenarios

#	Scenario	Expected	Status	Proof
1	Customer commits to paying today	PTP_TODAY	PASS	Asha Today - Promise To Pay Today
2	Customer provides a future PTP date	PTP_FUTURE	PASS	Ben Future - Promise To Pay
3	Customer states payment is complete	ALREADY_PAID	PASS	Carla Paid - Already Paid
4	Customer requests callback	CALLBACK_SCHEDULED	PASS	Deepak Callback - Callback Scheduled
5	Refuses due to financial difficulty	RTP_FINANCIAL	PASS	Elena Hardship - Financial Hardship
6	Disputes EMI amount	DISPUTE_CHARGES	PASS	Farah Dispute - Charge Dispute
7	Third party answers	THIRD_PARTY	PASS	Gita Account - Third Party
8	Changes language	PTP_FUTURE	PASS	Hari Bilingual - Hindi request plus future PTP
9	Disconnected, no clear outcome	DSCN	PASS	Iris Disconnected - Disconnected
10	Duplicate post-call webhook	Idempotent duplicate	PASS	Same X-Webhook-Id returns duplicate; no second update
11	Invalid initial request	HTTP 422	PASS	Serialization defect found during live run and fixed in commit df76400
12	Gnani trigger fails or times out	HTTP 502 / 504	PASS	Mock failure/timeout store trigger_failed; parametrized API test

TEST EXECUTION NOTE

The first mandatory-suite run completed scenarios 1-10 and exposed an error-serialization defect during scenario 11. The defect was corrected in commit df76400; the runner was also hardened to retain non-JSON provider responses. The dashboard overview visibly contains all scenario 1-9 records.

5. Acceptance criteria

Criterion	Status	Assessment
FastAPI initiation	PASS	Complete FDE request accepted, validated, stored, and returned as triggered in mock mode.
Customer receives call from configured Gnani agent	PASS	Verified live call, recording, and transcript from the configured Gnani agent. The demonstrated call was started from the Agent Console.
Prisma ASR, Timbre 2.5, Evon LLM	PARTIAL	Prisma v2.5 product and Evon v2.0 Fast are evidenced. The agent uses Timbre G v1.0 because it was the only TTS model available in this Agent Console account; Timbre 2.5 was not offered.
Meaningful multi-turn conversation	PASS	Live recording and transcript captured before Agent Console credits were exhausted.

Criterion	Status	Assessment
Post-call trigger received	PARTIAL	Simulated authenticated webhook received and stored; exact automatic provider webhook contract not captured.
Stage code and reason stored	PASS	MongoDB and dashboard evidence.
Outcome visible on dashboard	PASS	Scenario overview and completed Viji medical-hardship detail.
Invalid, failed trigger, duplicate handled	PASS	Explicit handlers and automated tests.
Runnable using README	PASS	Live Render URLs lead; localhost instructions are optional.

6. Why I used mock mode

The FDE document describes an outbound call-trigger API that receives customer data and invokes Gnani. My available Gnani Agent Console account did not expose a trigger endpoint, provider API key, or body template. The Initial Message callback I observed instead sent session metadata with blank phone fields and omitted customer and EMI variables.

Item	Value
Observed callback fields	call_id, flow_id, organization_id, environment, user_id, sender_id, blank phone_number/mobile
Missing business fields	customer_name, customer_id, loan account, EMI amount, due date, preferred language
Credits limitation	My Agent Console voice-call credits were exhausted; visible API-playground credits belong to a separate API product.
Demonstration fallback	I used the FastAPI mock trigger to store the complete request/provider ID, then completed it with a simulated authenticated webhook.
Not claimed	No claim that mock mode placed a phone call or called an unavailable Gnani trigger endpoint.

IMPORTANT DISTINCTION

Pre-call variables configured inside the agent can guide its prompt, but the captured Initial Message request proves that those values were not serialized into this callback. A production integration must receive a documented Gnani trigger contract or resolve customer data from a trusted system using a stable identifier.

7. Stage-code and idempotency logic

Stage code	Meaning	Dashboard group
PTP_TODAY	Promise to pay today	Promise to pay
PTP_FUTURE	Promise on future date	Promise to pay
ALREADY_PAID	Payment already completed	Paid
CALLBACK_SCHEDULED	Requested callback	Follow-up
RTP_FINANCIAL	Refused due to financial hardship	Other completed
RTP_MEDICAL	Unable to pay due to medical hardship	Other completed
DISPUTE_CHARGES	EMI/charge dispute	Other completed
THIRD_PARTY	Third party answered	Other completed
DSCN	Disconnected without outcome	Unreachable
trigger_failed	Provider trigger failed or timed out	Failure

Post-call delivery requires X-Webhook-API-Key. X-Webhook-Id is used as the delivery key; if omitted, the backend derives a SHA-256 fingerprint. The repository performs an atomic conditional update only when the delivery ID is absent. Repeating the same event returns duplicate and does not apply the outcome twice.

8. Submission requirement inventory

Requirement	Status	Location / evidence
Source code	PASS	GitHub repository
Gnani configuration/export	PASS	Verified Agent Console configuration with screenshots covering agent details, languages, model, voice, analytics fields, and conversation logs
Bot prompt and flow	PASS	Agent screenshots and repository documentation
FastAPI application	PASS	backend/app
Dummy dashboard	PASS	frontend plus live Render URL
Database schema	PASS	README data model and MongoDB screenshot
Postman/cURL	PASS	postman/CollectFlow.postman_collection.json and README
.env.example	PASS	Repository root
Dockerfile	PASS	Backend and frontend Dockerfiles
docker-compose.yml	PASS	Repository root
README setup	PASS	Live deployment first; local setup optional
Architecture diagram	PASS	This document and docs/architecture.md
Sample recordings	PASS	submission/recordings MP3
Sample webhook payloads	PASS	Postman collection, README, scenario runner
Dashboard screenshots	PASS	submission/evidence/dashboard
Stage-code logic	PASS	This document and code
Mandatory test results	PASS	Dashboard overview plus automated runner

9. Bonus requirements

I list only the bonus items that I passed or partially demonstrated.

Bonus item	Status	Assessment
Dockerised deployment	PASS	Two Dockerfiles and Compose definition
Call analytics charts	PARTIAL	Summary KPI cards exist; charting is not implemented
API authentication	PARTIAL	Webhook key authentication implemented; dashboard/operator auth not implemented
PII masking	PASS	Phone numbers are masked on dashboard rows and detail headers
Detailed audit logs	PARTIAL	Structured logs and webhook IDs; full audit ledger not implemented
Cloud deployment	PASS	Render plus MongoDB Atlas
CI/CD automated tests	PARTIAL	Automated tests exist and Render deploys from Git; no captured CI workflow evidence

10. Security and privacy

- Secrets remain in environment variables and are excluded from Git.
- Post-call webhooks require a constant-time compared API key.
- Delivery IDs make webhook retries idempotent and auditable.

- The frontend never receives MongoDB credentials.
- Dashboard phone numbers are masked; selected PDF screenshots avoid full customer phone numbers.
- Any webhook credential displayed during development should be rotated before submission.
- Demo records use synthetic customers and outcomes.

11. Production-readiness plan

FREE-TIER DEPLOYMENT OPERATION

I currently use Render's free tier, so the API and dashboard can become inactive after an idle period. Before demonstrating the project, I deploy the latest commit to bring the services back to a fresh live state and then refresh the dashboard. The first backend request can take about 50 seconds while the free-tier service starts.

Area	Recommended action
Provider contract	Obtain Gnani trigger URL, auth scheme, IP ranges, payload schema, retry behavior, and signed webhook contract.
Data enrichment	Resolve customer/EMI data from the system of record using a stable ID; do not trust session-only callbacks.
Authentication	Add operator identity, role-based access, session expiry, and least-privilege roles.
Webhook security	Prefer signed payloads with replay window, rotate keys, and alert on auth failures.
Reliability	Queue provider calls/webhooks, add exponential retry, dead-letter handling, and scheduled workers.
Observability	Add correlation IDs, structured audit events, metrics, traces, alerts, and provider dashboards.
Privacy	Encrypt sensitive fields, define retention/deletion, restrict Atlas access, and review logs for PII.
Quality	Run backend, frontend, contract, and end-to-end tests in CI before deploy; use staging gates.
Scale	Add pagination, indexes, rate limits, pool tuning, and backpressure for bulk campaigns.

12. Evidence appendix

I retained the following figures under submission/evidence to make this PDF self-contained. The dashboard filtering evidence uses the corrected, working Promise-to-pay result.

Figure 2. Mandatory scenario records visible on the deployed dashboard

Nine distinct customer outcomes demonstrate the completed business scenarios.

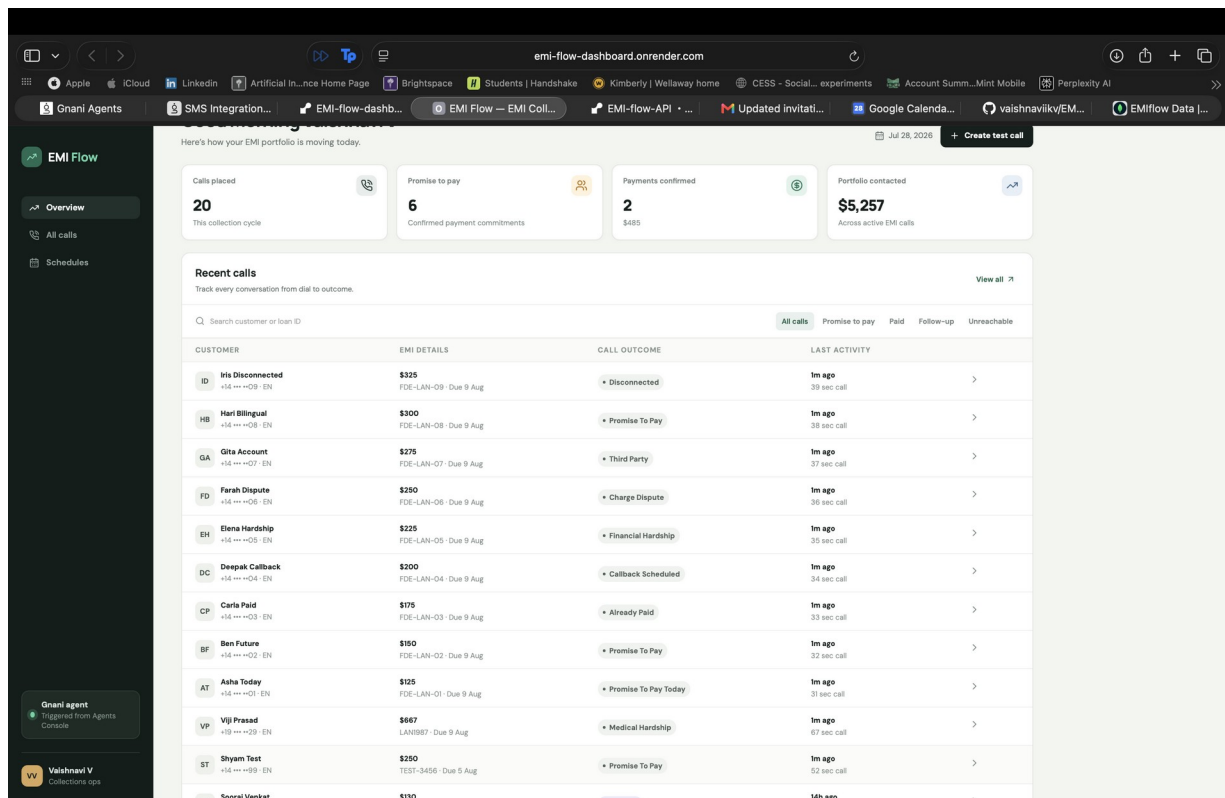


Figure 3. Corrected Promise-to-pay dashboard filter

The working filter returns three matching completed calls.

The screenshot displays the EMI Flow dashboard interface. The top navigation bar includes the EMI Flow logo and a sidebar with 'Overview', 'All calls', and 'Schedules'. The main content area is titled 'Good Morning Vaishnavi V' and shows a 'Collections Overview' with four key metrics: 'Calls placed' (11), 'Promise to pay' (3), 'Payments confirmed' (1), and 'Portfolio contacted' (\$3,232). Below these metrics is a 'Recent calls' section with a search bar and a table of call records. The table has columns for 'CUSTOMER', 'EMI DETAILS', 'CALL OUTCOME', and 'LAST ACTIVITY'. The 'CALL OUTCOME' column is filtered to show only 'Promise To Pay' calls. Three calls are listed, all with a 'Promise To Pay' outcome. The bottom of the dashboard shows a 'Gnani agent' status and a 'Vaishnavi V' profile card.

EMI Flow

Overview

Good Morning Vaishnavi V

Here's how your EMI portfolio is moving today.

Jul 28, 2026 [+ Create test call](#)

Calls placed 11
This collection cycle

Promise to pay 3
Confirmed payment commitments

Payments confirmed 1
\$310

Portfolio contacted \$3,232
Across active EMI calls

Recent calls
Track every conversation from dial to outcome. [View all 71](#)

Search customer or loan ID

All calls **Promise to pay** Paid Follow-up Unreachable

CUSTOMER	EMI DETAILS	CALL OUTCOME	LAST ACTIVITY
ST Shyam Test +14 *** +99 - EN	\$250 TEST-3456 - Due 5 Aug	• Promise To Pay	1m ago 52 sec call >
DC Demo Customer +14 *** +24 - EN	\$250 DEMO-9878 - Due 31 Jul	• Promise To Pay	1d ago 32 sec call >
MR Maya Rivera +14 *** +84 - EN	\$250 LN-84021 - Due 30 Jul	• Promise To Pay	1d ago 52 sec call >

Showing 3 of 11 recent calls Data refreshed just now

Gnani agent
Triggered from Agents Console

VV Vaishnavi V
Collections ops

Figure 4. Completed medical-hardship call detail

Shows outcome summary, RTP_MEDICAL, duration, transcript, and verified webhook payload.

The screenshot displays the EMI Flow dashboard interface for a completed medical-hardship call. The dashboard is titled "CALL RECORD - SIM-VIJI" and shows the customer's name "Viji Prasad" with a phone number "+19 --- --29 - English - LAN1987".

Call outcome (67 sec):

Viji Prasad is currently unable to pay the \$667 EMI because of unexpected medical expenses. The customer did not commit to a payment date or request a specific callback time.

Disposition	Sentiment	EMI amount	Due date
RTP MEDICAL	Unknown	\$667	8 Aug 2026

Provider payload (Verified webhook):

```
{
  "speaker": "customer",
  "text": "I am sorry, but I cannot make the payment right now because I have unexpected medical expenses.",
  "timestamp_ms": null
},
{
  "speaker": "agent",
  "text": "I understand. Would you like to commit to a payment date or schedule a callback?",
  "timestamp_ms": null
},
{
  "speaker": "customer",
  "text": "I cannot confirm a date yet. I need some time until my medical situation is resolved."
}
```

Conversation (EN):

AI: Gnan agent
Hello Viji, I am calling regarding the \$667 EMI for your loan account ending in 1987. Are you able to make the payment by the due date?

Viji Prasad
I am sorry but I cannot make the payment right now because I have unexpected medical expenses.

V: Viji Prasad
I cannot confirm a date yet. I need some time until my medical situation is resolved.

AI: Gnan agent
Understood. I will record that you are currently unable to pay due to medical hardship.

Figure 5. MongoDB Atlas completed-call document

The record includes stage code, webhook ID, transcript, outcome, reason, PTP date, and provider ID.

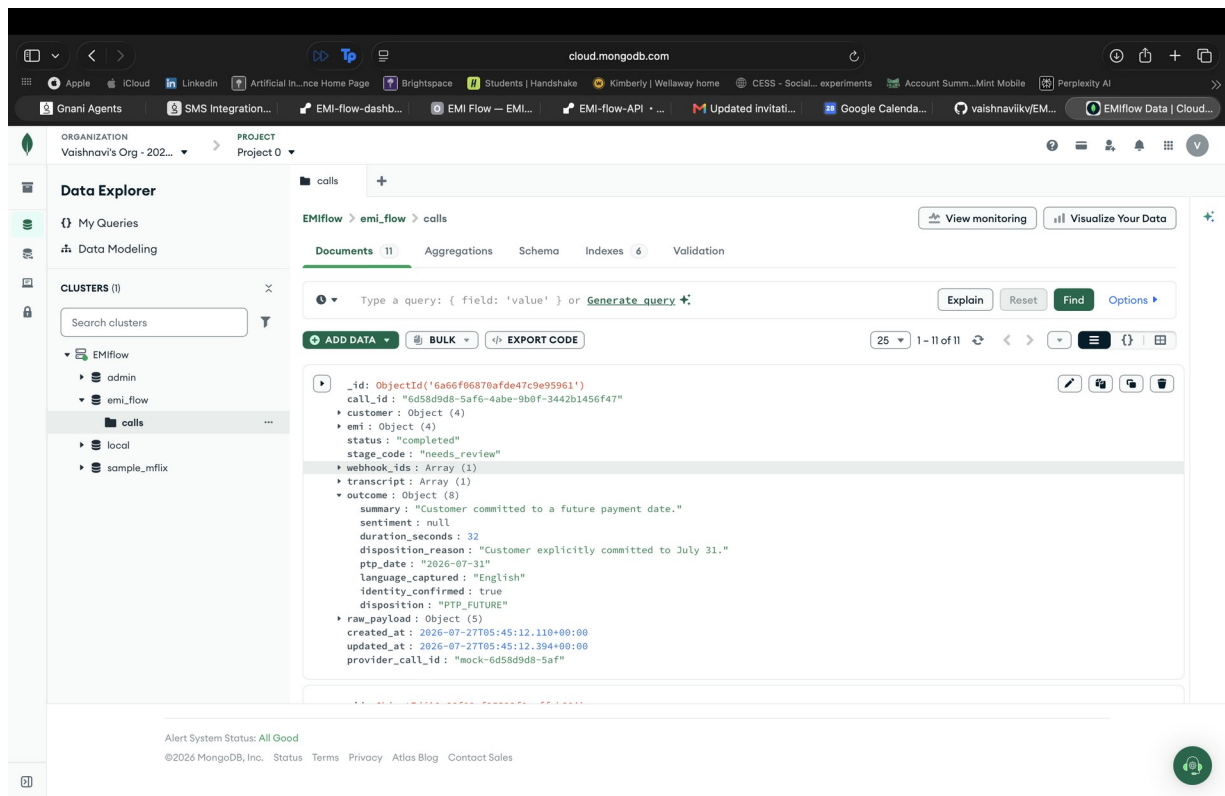


Figure 6. FastAPI mock-call initiation form

The form is explicitly labeled as mock mode and states that no phone call is placed.

The screenshot displays the EMI Flow dashboard interface. A modal window titled "FDE DEMONSTRATION" and "Create a mock Gnani call" is open in the center. The modal contains the following fields and information:

- Customer ID:** CUST001
- Customer name:** Rahul Sharma
- Phone:** +1 9175550123
- Loan account:** LAN123456
- EMI amount:** (empty field)
- Currency:** USD
- EMI due date:** 28/07/2026
- Preferred language:** English

Below the form fields, a yellow banner states: "Mock mode: FastAPI validates and stores the request, generates the greeting and provider ID, but does not contact Gnani." At the bottom right of the modal are "Cancel" and "Create test call" buttons.

The background dashboard shows a "COLLECTIONS OVERVIEW" for "Good Morning Vaishnavi V". It includes a "Calls placed" section with a count of 11, a "Promise to pay" section with a count of 3, and a "Payments confirmed" section with a count of 1. A "Portfolio contacted" section shows a value of \$3,232. A "Recent calls" table lists various customer interactions with details like customer name, phone number, and status.

Figure 7. Gnani agent details and language-switch configuration

English/Spanish, region, time zone, description, and switch threshold are visible.

The screenshot shows the 'Gnani' application interface for configuring an agent named 'EMI Payment Collection Agent'. The interface is divided into a left sidebar and a main configuration area.

Left Sidebar:

- Build** (selected)
- Agents** (selected)
- Knowledge Base
- Workforce
- Settings (gear icon)
- Help (question mark icon)
- Logout (VV icon)

Main Configuration Area:

Configuration (selected) | Actions

Agent Details

Language: English (US) x Spanish x

Region: America

Time Zone: Pacific Standard Time (PST)

Description: Outbound AI voice agent for EMI payment reminders. The agent confirms customer identity, explains the EMI context, captures payment intent, handles objections and callbacks and generates a structured post-call disposition.

Language Switch Trigger Threshold: Minimum number of words the user must speak in another language before the bot checks for a language switch. 2

Language Switch Prompt: Add additional information or prompt specifically for different languages. **Custom Mode** (selected)

Custom Mode Prompt: In Custom mode, you can write your own instructions for how the agent should respond when a user requests a language switch.

Buttons: Save, Test, Conversation logs, Get Started

Figure 8. Gnani Evon LLM configuration

The captured model is Gnani Evon v2.0 Fast.

The screenshot shows the 'Build' interface for an agent named 'EMI Payment Collection Agent' on the 'app.gnani.ai' platform. The left sidebar contains navigation options: Agents, Knowledge Base, Workforce, and a settings gear. The main area is divided into 'Configuration' and 'Actions' tabs. Under 'Configuration', there are sections for 'System Prompt', 'Agent Details', 'Conversation Flow', 'Transcriber', 'LLM Model', 'Voice', 'Analytics', and 'FAQ Answers'. The 'LLM Model' section is active, showing 'Model Settings' where the provider is 'Gnani' and the model is 'Gnani Evon v2.0 Fast'. Below this is the 'Knowledge Base' section with a 'Select Knowledge Base' dropdown and a note to connect a KB for better accuracy. The 'Temperature' slider is set to 0.2, and the 'Max Tokens' slider is set to 300. A 'Get Started' button is at the bottom right.

Build

Agents

Knowledge Base

Workforce

EMI Payment Collection Agent

Configuration Actions

System Prompt

Agent Details

Conversation Flow

Transcriber

LLM Model

Voice

Analytics

FAQ Answers

Model Settings

Choose the AI model provider for your agent.

Select Provider * Gnani

Select Model * Gnani Evon v2.0 Fast

Knowledge Base

Connect a knowledge base for your agent to refer

Select Knowledge Base

Connect a KB with all required information to help agent answer questions more accurately

Temperature

Adjust creativity of your agent's response; higher values make it more random.

0.2

Max Tokens

Max input length in tokens (~3-4 chars or 3/4 of a word). Higher limits allow longer input but slower response.

300

Get Started

Figure 9. Gnani TTS configuration

The agent uses Timbre G v1.0 with Jenny because it was the only TTS model available in this Agent Console account; Timbre 2.5 was not offered.

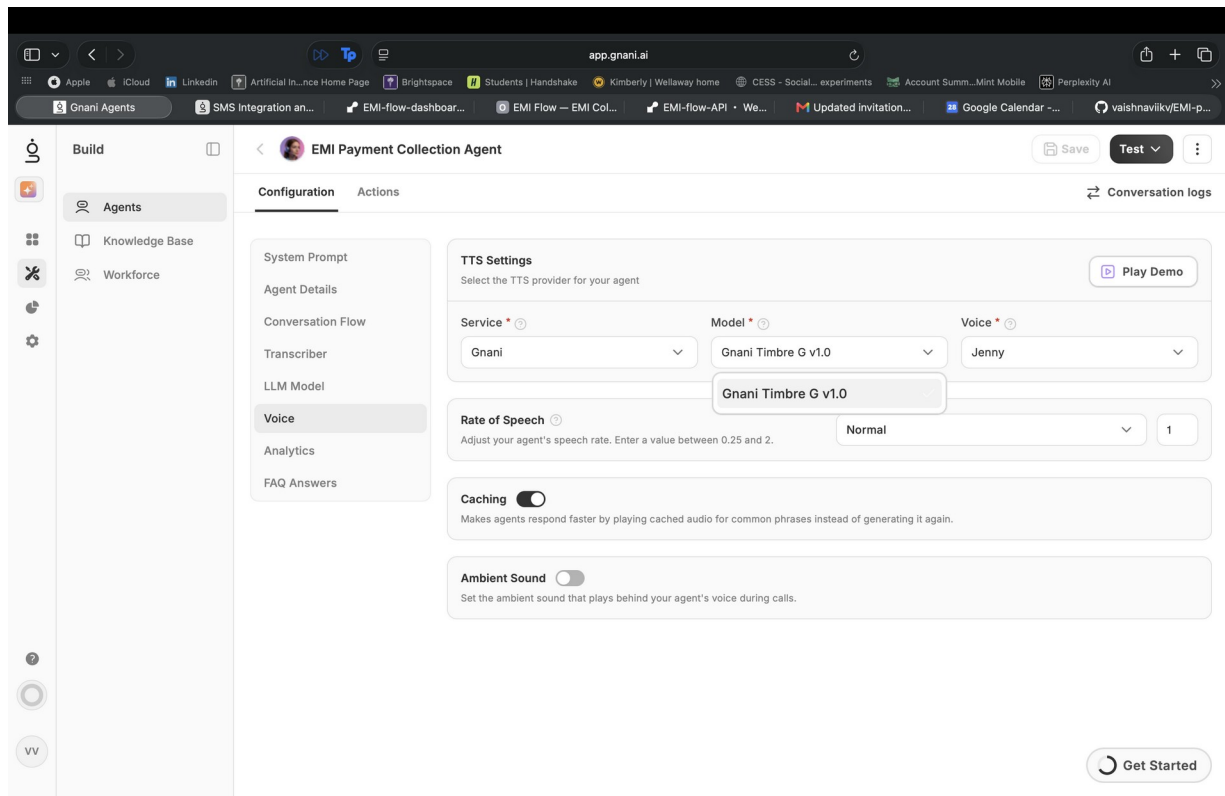


Figure 10. Structured post-call extraction fields

Disposition reason, summary, PTP date, callback time, and language are configured.

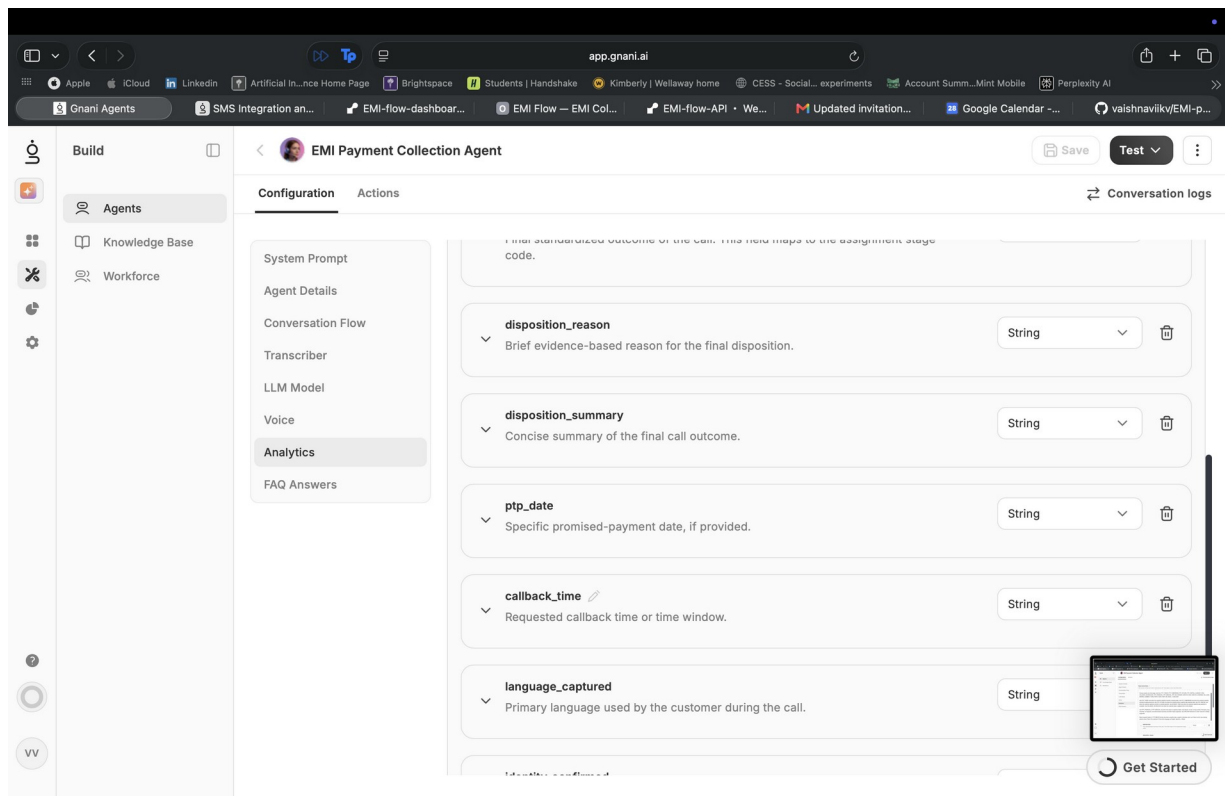


Figure 11. Real Gnani multi-turn voice-test recording and transcript

Conversation 9b8a1885-efd3-4d9a-9fdc-367de019e116 lasted 1m08s.

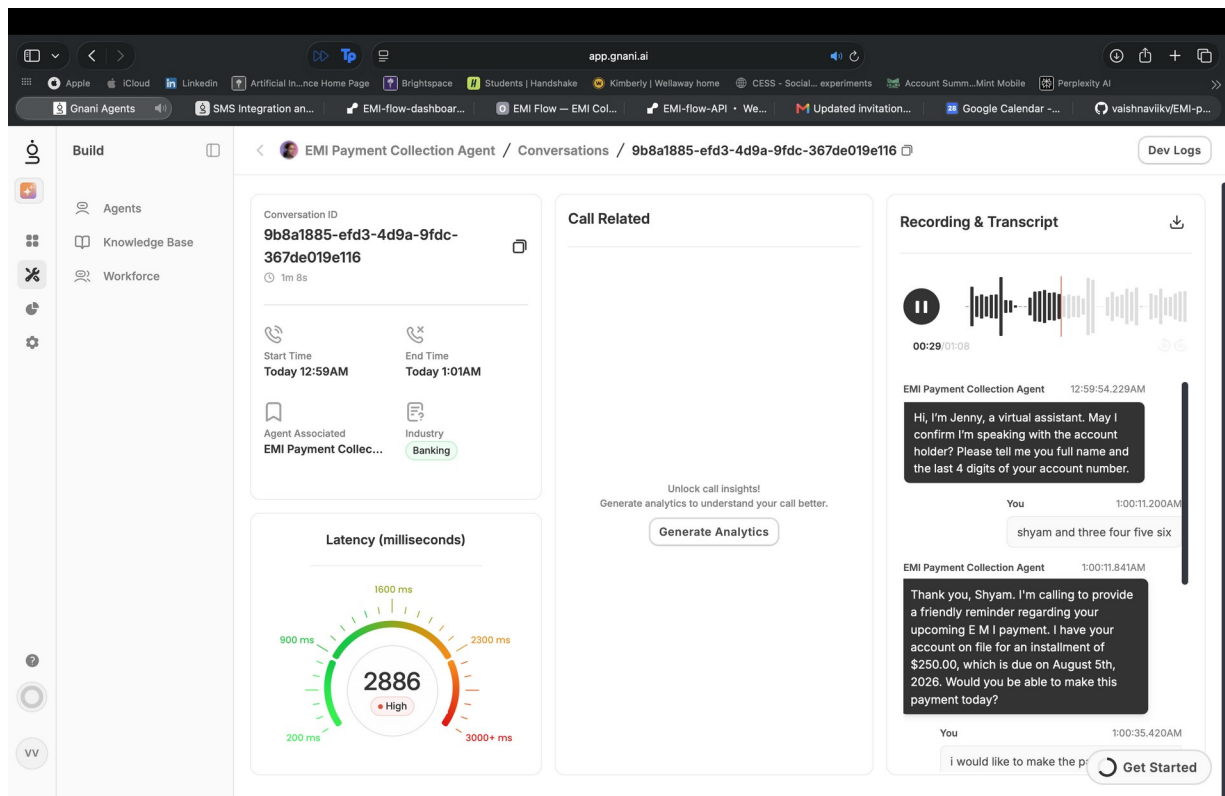


Figure 12. Gnani Prisma API playground and separate API credits

This proves Prisma v2.5 API access, not Agent Console call credits or agent transcriber selection.

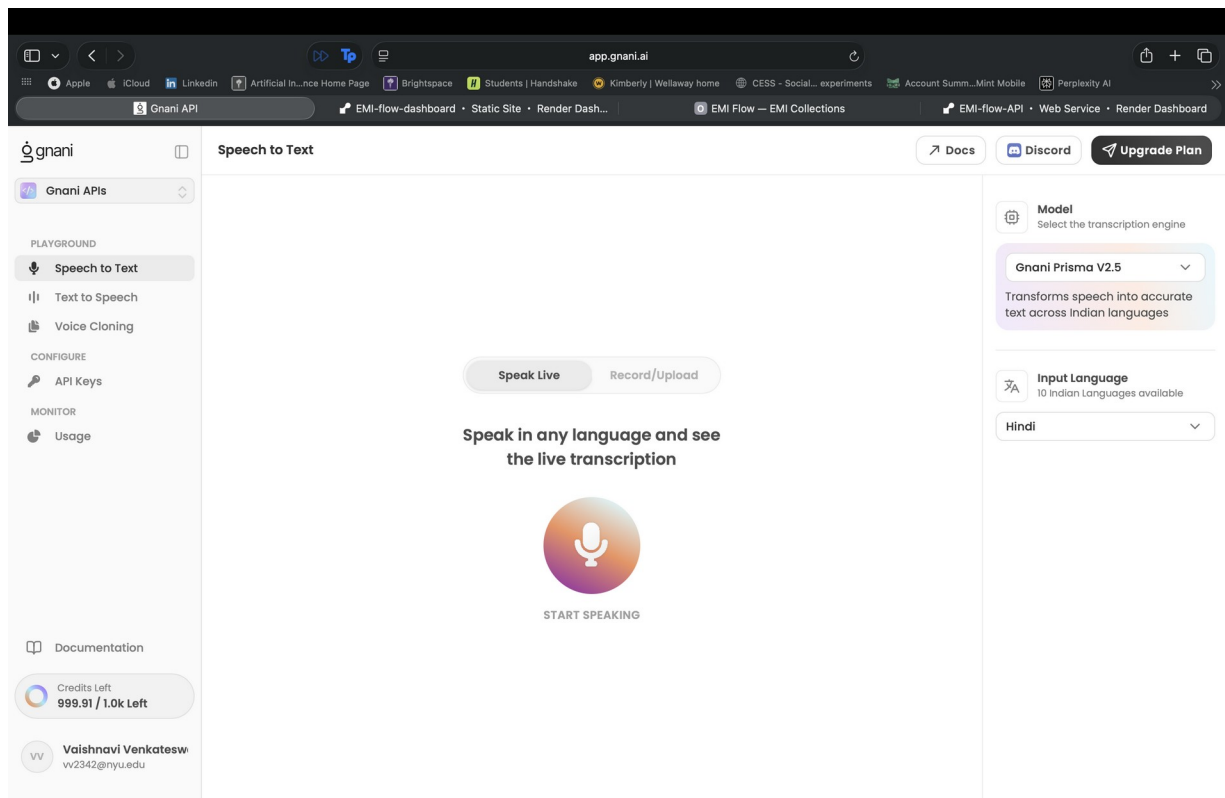


Figure 13. FastAPI deployed as a live Render Web Service

The backend is publicly reachable over HTTPS.

The screenshot displays the Render dashboard for a project named "EMI-flow-API". The service is a "WEB SERVICE" using "Docker" and is currently in a "Free" tier. The service ID is "srv-d9yv0rbm8hqs73b7idf0". The service is live, as indicated by the "Live" status and the "Your service is live" message. The logs show the deployment process, including the setting of "WEB_CONCURRENCY=1" and the successful startup of the application. The logs also show the application running on "http://0.0.0.0:8000" and the "HEAD / HTTP/1.1" 404 Not Found response.

Environment: EMI-flow-API

Events

Settings

MONITOR

Logs

Metrics

MANAGE

Environment

Shell

Scaling

Previews

Disk

One-Off Jobs

Changelog

Invite a friend

Contact support

Render Status

WEB SERVICE

EMI-flow-API

Docker Free Upgrade your instance →

Connect Manual Deploy

Service ID: srv-d9yv0rbm8hqs73b7idf0

vaishnaviky / EMI-payment-collection-agent main

https://emi-flow-api.onrender.com

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more. Upgrade now

July 28, 2026 at 2:34 PM Live

bc137b3 Accept Gnani session payloads and recover calls

All logs Search logs

Jul 28, 2:33 PM - 2:36 PM PDT

Jul 28

02:34:36 PM #14 DONE 0.2s

02:34:40 PM ==> Deploying...

02:34:40 PM ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance

02:34:52 PM [n5xwp] INFO: Started server process [1]

02:34:52 PM [n5xwp] INFO: Waiting for application startup.

02:34:52 PM [n5xwp] INFO: Application startup complete.

02:34:52 PM [n5xwp] INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

02:34:53 PM [n5xwp] INFO: 127.0.0.1:52852 - "HEAD / HTTP/1.1" 404 Not Found

02:35:01 PM ==> Your service is live 🎉

02:35:01 PM [n5xwp] INFO: 35.247.124.215:0 - "GET / HTTP/1.1" 404 Not Found

02:35:01 PM ==>

02:35:01 PM ==> ///

Figure 14. Dashboard deployed as a live Render Static Site

The production Vite build completed and the site is live.

The screenshot displays the Render dashboard interface for a project named "EMI-flow-dashboard". The left sidebar contains navigation options: Dashboard, EMI-flow-dashboard, Events, Settings, MONITOR, Metrics, MANAGE, Environment, Previews, Redirects/Rewrites, Headers, Changelog, Invite a friend, Contact support, and Render Status. The main content area shows the service ID "srv-d9jv8e3m8hqs73b87r0", the agent "vaishnavikv / EMI-payment-collection-agent", and the URL "https://emi-flow-dashboard.onrender.com". A deployment event is listed for "July 28, 2026 at 2:53 PM" with a "Live" status. Below this, a log viewer shows the build process, including the command "vite v6.4.3 building for production...", the transformation of 1582 modules, and the rendering of chunks. A table lists the build artifacts and their sizes:

File	Size	gzip Size
dist/index.html	0.55 kB	0.34 kB
dist/assets/index-B0X6jW7P.css	11.38 kB	3.30 kB
dist/assets/index-DuHma2ce.js	222.70 kB	69.01 kB

The logs conclude with the message "Your site is live". At the bottom, a note suggests using the Render CLI, MCP Server, or log stream integration for better log management.

13. Project summary and transparency

CORE APPLICATION: BUILT AND DEMONSTRATED

I implemented and evidenced FastAPI validation, pending-call persistence, post-call idempotency, structured outcome storage, MongoDB persistence, dashboard visibility, search/filter behavior, failure paths, Docker packaging, and cloud deployment.

PROVIDER INTEGRATION: DOCUMENTED LIMITATIONS

I demonstrated a real Gnani call, conversation, and recording. I could not fully demonstrate provider-triggered outbound calling, exact automatic webhook delivery, or the agent Prisma transcriber selection. I used Timbre G v1.0 because it was the only TTS model available in my Agent Console account; Timbre 2.5 was not offered.

I created this project as a technically complete and reproducible application-layer demonstration. The deployed demo uses Render's free tier, so I deploy the latest commit and refresh the dashboard before a live demonstration when the services have been inactive. To complete the remaining provider-specific integration, Gnani must issue the outbound-trigger credentials and restore Agent Console call credits. I can then run one final live call using the same call_id through initiation, conversation, webhook, MongoDB update, and dashboard display.

TRANSPARENCY NOTE

The limitation is caused by unavailable trigger credentials and exhausted Agent Console credits, not hidden by the submission.